

Protocols and Layer Reference Models

Concept of Reference Models:

- Each layer provides a service to the layer above it, so the upper layer can focus on its own tasks without worrying about the details of how the lower layer performs its job. Each layer handles a specific part of the process, like delivering data reliably, routing it, or converting it into signals.
- When an upper layer wants to send data to the directly lower layer it uses a SAP which is the entry for the data, so simply the above layer calls the SAP of the lower layer to send the data to the layer.
- If a layer is to be excluded (not implemented) the layers above and below should be able to communicate effectively with no disruption. For an example the application layer send data directly to the internet layer skipping the transport layer. The internet layer should be implemented in a way that can read and work on the data formatted in the application layer instead of that of the transport layer. That does not mean that any layer should do the job of the transport layer, the headers of the transport layer and the data formatting is totally ignored and not done by any layer later in the process.
- Details of the service (what the layer does) are ruled by a respective protocols (how its done the rules, procedures to follow to achieve this service).
- Transparency in wording and duties: in wording means that the layers communicate using simple direct messages that does not explain much. Transparency for duties means each layer does not what protocols or how the other layer works internally at all.
- Most popular reference modules for data communication:
 - OSI(an abstract module not implemented in real life but used as main reference)
 - TCP/IP used and implemented in real life data communication
 - Hybrid: Models that combine aspects of OSI and TCP/IP for teaching or design purposes.

Protocol:

A protocol is basically a set of rules or agreements that two communication partners follow so that they can understand each other and communicate correctly. These communicating partners can be 2 layers, 2 OS, 2 processes. Without protocol the communication would be chaotic since the communicating partners might interpret the message differently.

What does the protocol includes:

- How to start a communication session and how to end properly
- Synchronisation of service provider and taker (or sender and receiver); the protocol defines the synchronization between the layers, whether the data should be sent in a half duplex, the speed of which data is sent to the receiver

not to overwhelm the service provider. If the communication is horizontal we might need for acknowledgement this is also defined in the synchronization protocol.

- Formats and encoding of messages; defines how the data should be formatted and decode/encoded.
- detecting and treating transmission errors; the protocols define ways to catch transmission errors if the message or data frame is not sent or corrupt, the protocol also defines how to handle it if it happened.

Protocol includes:

- **Syntax:** The format, order of the communicating message and the layout. Simply how the message should be grammatically formatted. For an example, in the application layer the http request should include GET as the first word In the message, this GET means that the first word of the message should be the method.
- **Semantics:** What does each part of the message mean, what actions should be taken based on the message that the layer has received. So the semantics means once the layer gets the method as GET it know it should return an html media

TCP/IP Reference Model:

It's a network architecture that separates between the application (the actual application that you deal with like a web browser not the application layer) and the media (the network card, the optic fiber cable not the link layer) into 4 layers.

N.B: the link layer sets hypothetically above the media; cables.

Application is above the layers while media is below the architecture layers.

4th layer Application: HTTP(s), FTP, SMTP, POP, DNS, SSH, Telnet

3rd layer Transport: TCP, UDP

2nd Layer Internet: IP (IPv4, Ipv6), ICMP, IPsec, IPX

1st Layer Link: Ethernet, WLAN; ATM, FDDI, PPP, Token Ring.

Vertical communication:

Each layer adds or removes a header and a tail if a tail exists.

The link layer receives signals, translates it into data frames, then remove the link layer heads and trailers and forward the entire message to the network layer. And the opposite. The link layer decide which protocol to format the message based on which physical media it has.

Explaining the flow with an example:

A user enters a url or clicks a button on a web browser, this click or url is sent then the Application layer, the application layer transform this click or url into http request (for instance) and send this http message to the transport layer. Transport layer receives the http message then adds the tcp header and forward the new message to the internet layer. The internet layer adds the ip header and transfers the message into the link layer. In a reverse situation the internet layer has now a message, it strips the ip out and send the tcp http together to the transport layer. The transport layer strips the tcp header and send the http message to the application layer. The application layer send the data to be shown on the browser.

Horizontal communication:

Its not how it actually works in real life, again its not an actual implementation or a real life flow of data communication. What it means is that the layers act if the communicate directly to each other by writing a message that the same layer in the crossponding system can read and understand. So link layer send the signals in a way that the link layer on the other side can read. The reply of the link layer on the other will also reply in a way that the first link layer can understand. The application layer on the first system would write the message in a way the other application layer would understand. So it seems like each layer is communicating to its crosspondate.

Hybrid OSI TCP/IP model:

It's called hybrid cause it has a combination from the "conceptual" OSI model and the TCP/IP model.

It splits the link layer into 2 layers (data link, physical layer)

The Link layer is renamed to be called data link layer; it does the exact same job as before.

The physical layer is simply the media (cables , network cards, ethernet)

Why?:

The 5-layer model exists mainly for clarity and precision, not because TCP/IP itself needs it.

It splits responsibilities that were combined in the old Link layer.

It helps teaching, comparing to OSI, and troubleshooting.

Data Transmission

Electrical Basic:

Signals in cables travel as electric current that has current (I) and the voltage (V) and we have 2 types of electric current or flow, AC alternating where the current and voltage changes over time and repeat itself and we have DC Direct current where the I and V are constant.

AC = Alternating Current. Current I (t), Voltage U (t), t here means function of time, changes periodically during time.

DC = Direct current, Current I = Constant, and Voltage U = constant.

Frequency F is number of oscillations per second, this means the amount of time that is need for the wave to start from 0 and go to its peak then go to its dip and back again to the 0.

Frequency $F = 1 / \text{Time (T)} = F$. example $F = 1 / 0.02 = 50 \text{ HZ}$.

Period T – Time duration it takes for a complete cycle of the waveform to occur.

$T = 1/F$ with $[T] = 1\text{s}$. another example for clarity; $F = 50 \text{ HZ}$. The $T = 1 / F = 1 / 50 = 0.02$ second

Real life frequencies:

fnet = 50Hz, ftone_a = 440Hz, fbaseWiFi1 = 2.4Ghz, fbaseWiFi2 = 5Ghz

AC Graphical Representation:

Sinusoidal:

A sinusoidal AC graph is a time-domain graph that represents how either the current I(t) or the voltage V(t) of an AC signal changes over time. It uses a sine function because AC oscillates smoothly, and the graph lets us observe the peak (maximum) and dip (minimum) values, which are defined by the signal's amplitude.

Rectangular:

rectangular wave jumps between up and down directly, where up represents the one and down represents the zero, it can include many frequencies so the up or the down can look smaller or bigger depending on the frequency, it can stay up for a while when many consecutive ones are being sent or the opposite for the zeros. did i do any misconceptions

Binary Data vs. Signal Transmission:

- **Binary Data:** is the data represented by sequence of zeros and ones.
- The data is transferred in computer networks on a higher level logically. Means lets assume between the 4 layers of the tcp/ip they all use binary encoding until it reaches the physical layer then its not binary any more but electricity.
- Signal transmission is always based on analogue signals (signal that varies over time), its based on the voltage or current amplitude (signal strenght, peak and dip). We can split this analogue into 3 types. Electrical (cables), wireless (radio), optic (fibers).
- **Signals are subject to physical laws (since they are just electric flow):**
 - **Attenuation (all):** signals get weaker as they travel further.
 - **Reflections (electromagnetic waves, optical):** Signals bounce back when they face an obstacle. In optical this can happen if for instance the fiber cable is bent too sharply the signal can hit it and bounce back.
 - **Shadowing (electromagnetic waves):** Signals get weaker or blocked whenever they face an obstacle. (obstacle can be a wall).
- **A problem; physical limitations of bridgeable distances without signal amplification:**
 - How far can a signal (e.g., electric pulse, light, or EM wave) travel before it becomes too weak or distorted to be received correctly, without using an amplifier or repeater?

Fourier Series:

Fourier series can only be used for periodic waves (signals).

A periodic function is a signal or waveform that repeats itself at regular intervals over time. Like; Sine wave itself, Square wave, Triangle wav, Any repeating digital signal pattern.

Fourier (the scientist PS: I hope he is in hell) discovered that any periodic function can be represented using a series of sine and cosine waves. Let that be periodic (triangle waves, square waves, etc...)

This helps break down any signal into sine and cosine waves that we can understand

The sine/cosine functions are multiple of a fundamental frequency: a fundamental frequency is the actual frequency of the entire signal, lets assume the signal has a frequency of 50HZ that means it makes 50 waves per seconds. The multiples are sine or cosine waves that has 2,3,4,5 Times the frequency of which the signal makes an entire wave.

The higher accuracy in approximation the higher frequencies are needed.

N.B: the amplitude of the multiple gets lower as the frequency gets higher

Fourier Transform and Analysis:

Fourier Transform is a natural extension of Fourier Series that works for all signals, not just repeating ones. It's the general version of the Fourier Series concept.

- **Time Domain:**

Shows the signal as it varies over time.

Example: a voltage that goes up and down, or a sound wave.

- **Fourier Transform:**

A mathematical tool that converts the time-domain signal into the frequency domain.

It analyzes the signal to find out which sine and cosine waves (frequencies) make up the signal and how strong they are.

- **Frequency Domain:**

Shows the frequencies present in the signal and their amplitudes.

Example: a signal in time might look complex, but in frequency domain you can see: 50 Hz, 150 Hz, 250 Hz components clearly.

Time Domain → Fourier Transform → Frequency Domain

Think of it as: time domain = what the signal looks like, frequency domain = what it's made of, and Fourier Transform = the bridge between the two.

Fourier analysis (Fourier Series / Transform) is a mathematical tool. It helps engineers understand and manipulate signals, whether they are analog (continuous) or digital (1s and 0s).

Examples for the uses of fourier analysis:

- Digital Signal Processing
- Digital Filtering
- i.e. MP3 compression/decompression

Fourier Series and Bandwidth:

Bandwidth: is the difference between the highest and the lowest frequency (range of frequencies) on the media (cable, fiber optic) with reliability. The higher the difference between the frequency edges the more data the media can transmit at the same time.

Bandwidth (in other words): width (amount) of frequencies in the frequency domain.

Bandwidth limitations by attenuation of higher frequencies (acts like a low pass filter); this means that the closer the frequency being transmitted to that of the maximum of the media (max included) (we are not talking about the range max-min) the weaker it gets while being transmitted compared to the low frequency.

Here something called low pass filter, means that the high frequency in the media gets weaker the closer they are to the edge that works like filtering the data.

Bandwidth: The highest amount of bits that can be transferred per second. Aka max bit rate.

Bit Rate vs Baud Rate:

Bit rate: number of bits transferred per timespan.

Timespan is usually one second so bps = bits per second; that how we measure the bit rate.

Baude rate: The number of symbols transferred per timespan

A symbol is a special signal transferred to the media that represents as many bits as the encoding type allow.

So we can think of the bit rate as a multiple of the baud rate where the bit rate = baud rate * bits per symbol

Ratio of bit rate to baud rate is define by the line encoding (type of encoding)

Sending: Bits → symbols → physical signals (higher layers use bits, symbols)

Receiving: Physical signals → symbols → bits

Latency:

Is the time needed for a message to travel through a network from one end of the network to the most distant end.

Latency = Propagation delay + Transmission delay + Waiting time + processing delay

Propagation delay = (distance)/(speed of light c * velocity factor) $c = 299\,792\,458$ m/s (speed of light)

Transmission delay = message size / bandwidth

Transmission delay: the time needed for the entire message to be placed from the first to the last bit on the media

Waiting time: is the time delay caused by intermediating devices, can also be called queuing time.

Bandwidth – Delay Product:

The product of bandwidth and delay (latency) corresponds to the maximum number of bits that can reside inside the line between sender and receiver.

Capacity of the link = bandwidth * latency (delay)

Example: A network with 100 Mbit/s bandwidth, and 10 ms latency => 100 000 000 Bits/s $\times 0.01$ s = 1 000 000 Bits